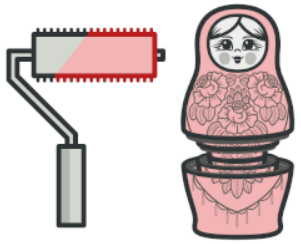
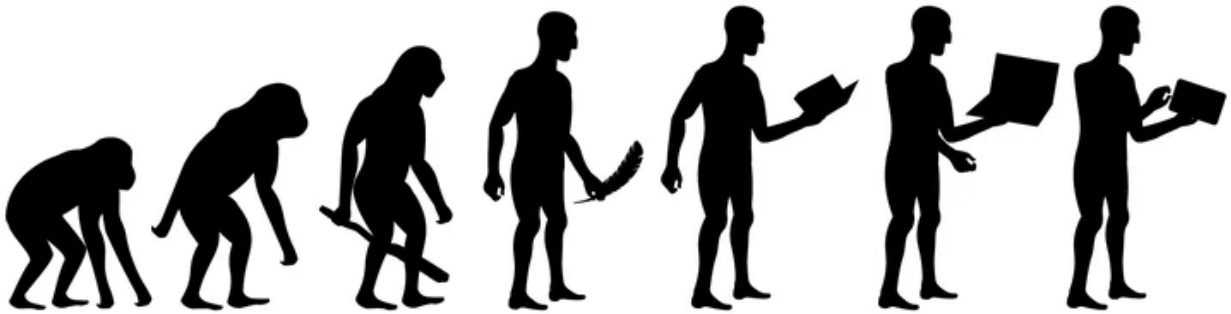
Структурные паттерны. Декоратор



Паттерн Декоратор
Decorator (Wrapper, Обёртка)

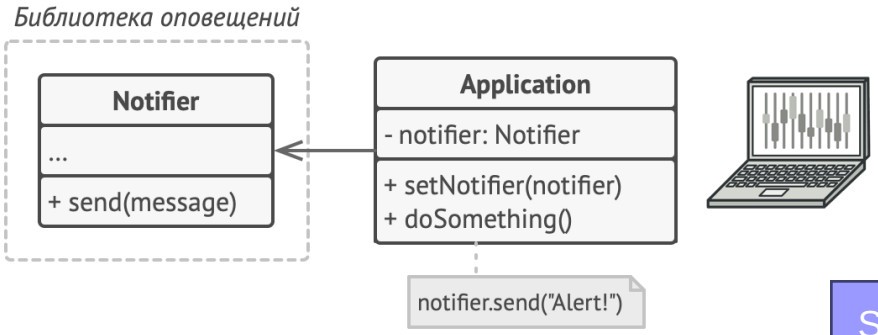


Паттерн **Декоратор** - динамически (в ходе выполнения программы) добавляет объекту новые возможности (*состояние и/или поведение*).

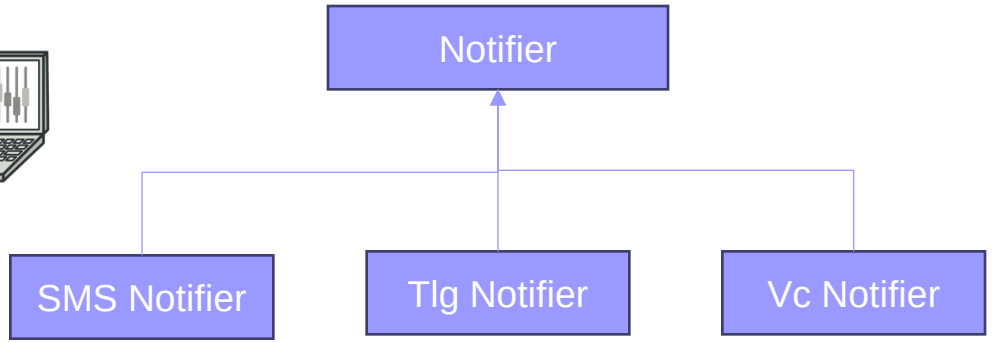
Композиция, используемая при реализации паттерна **Decorator**, является гибкой альтернативой наследованию (порождению подклассов) с целью *расширения функциональности.*

Проблема

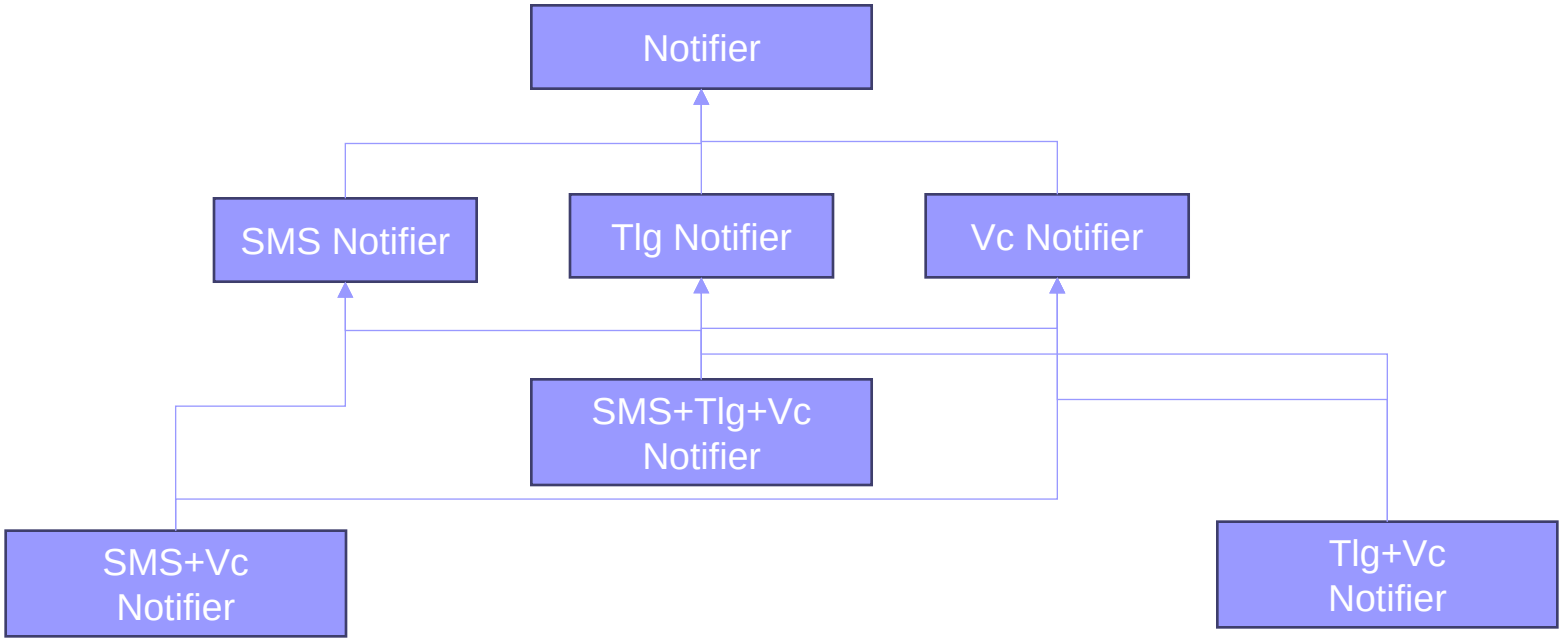
Библиотека оповещений



Использование библиотеки сторонними мессенджерами

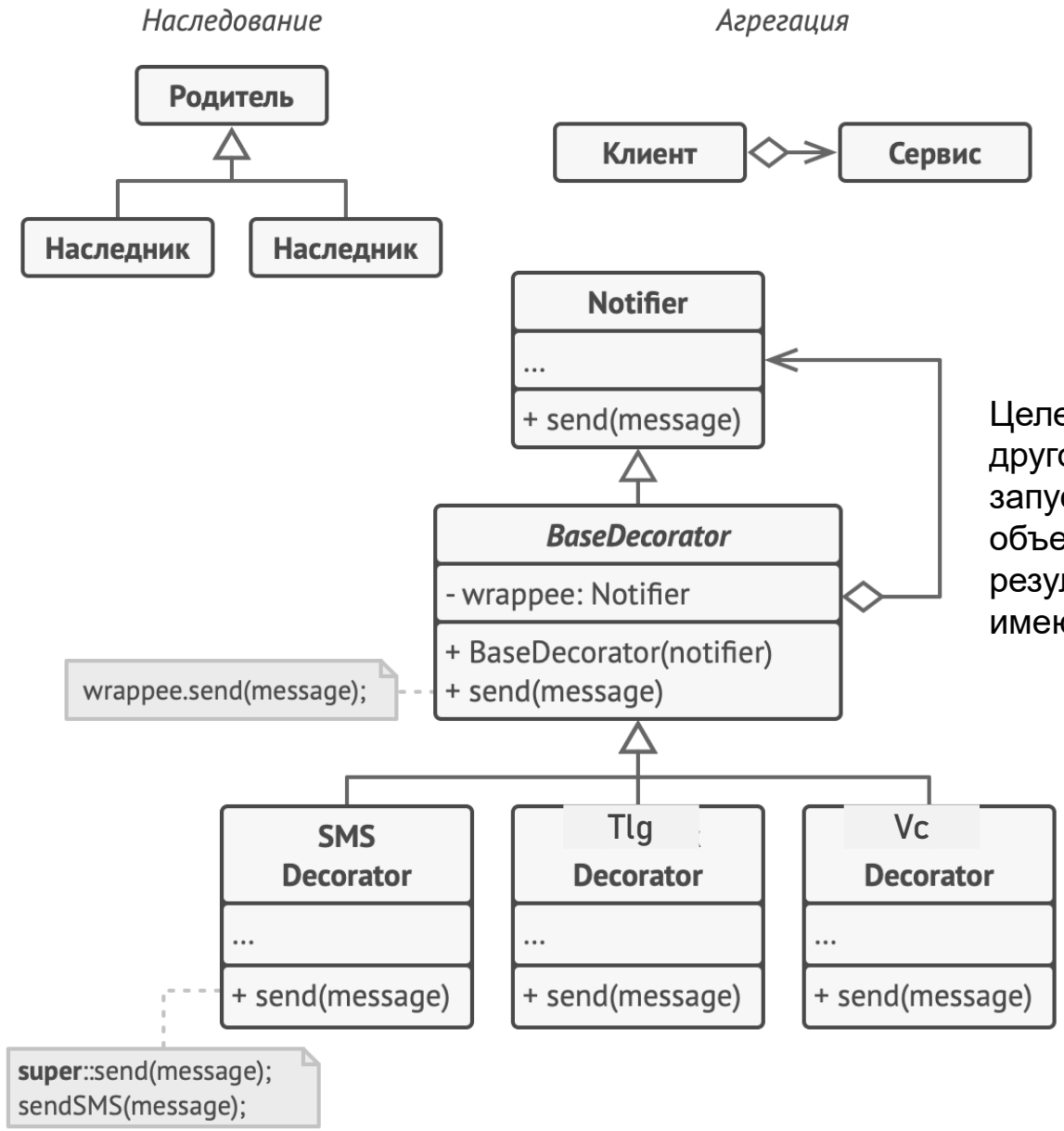


Комбинации классов оповещений



Решение

Замена наследования *агрегацией* или *композицией*



Альтернативное название Декоратора — **Обёртка**.

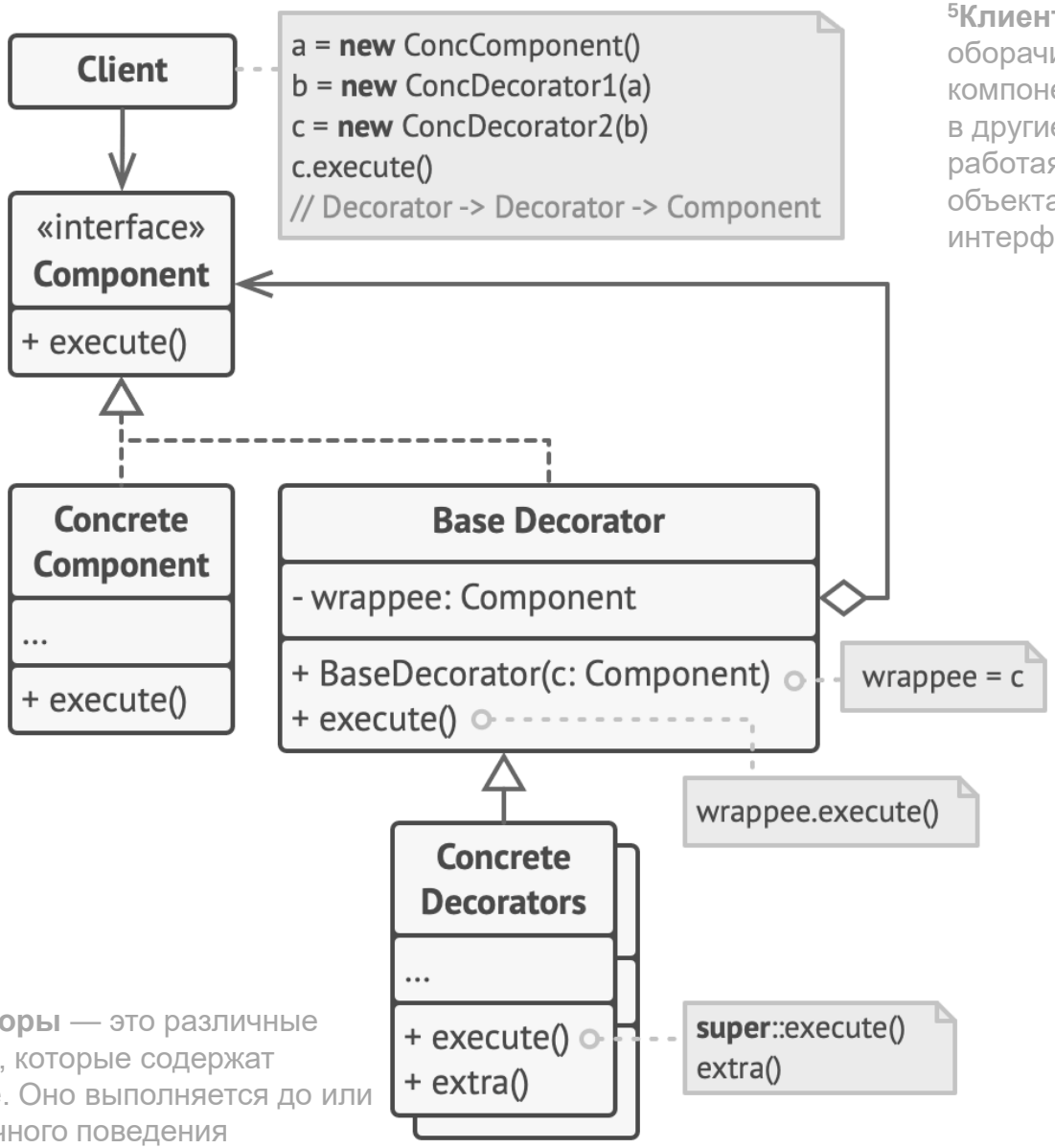
Целевой объект помещается в другой *объект-обёртку*, который запускает базовое поведение объекта, а затем добавляет к результату что-то своё. Оба объекта имеют общий интерфейс

Структура паттерна

[1]**Компонент** задаёт общий интерфейс обёрток и оборачиваемых объектов.

[2]**Конкретный компонент** определяет класс оборачиваемых объектов. Он содержит какое-то базовое поведение, которое потом изменяют декораторы.

[4]**Конкретные декораторы** — это различные вариации декораторов, которые содержат добавочное поведение. Оно выполняется до или после вызова аналогичного поведения обёрнутого объекта.

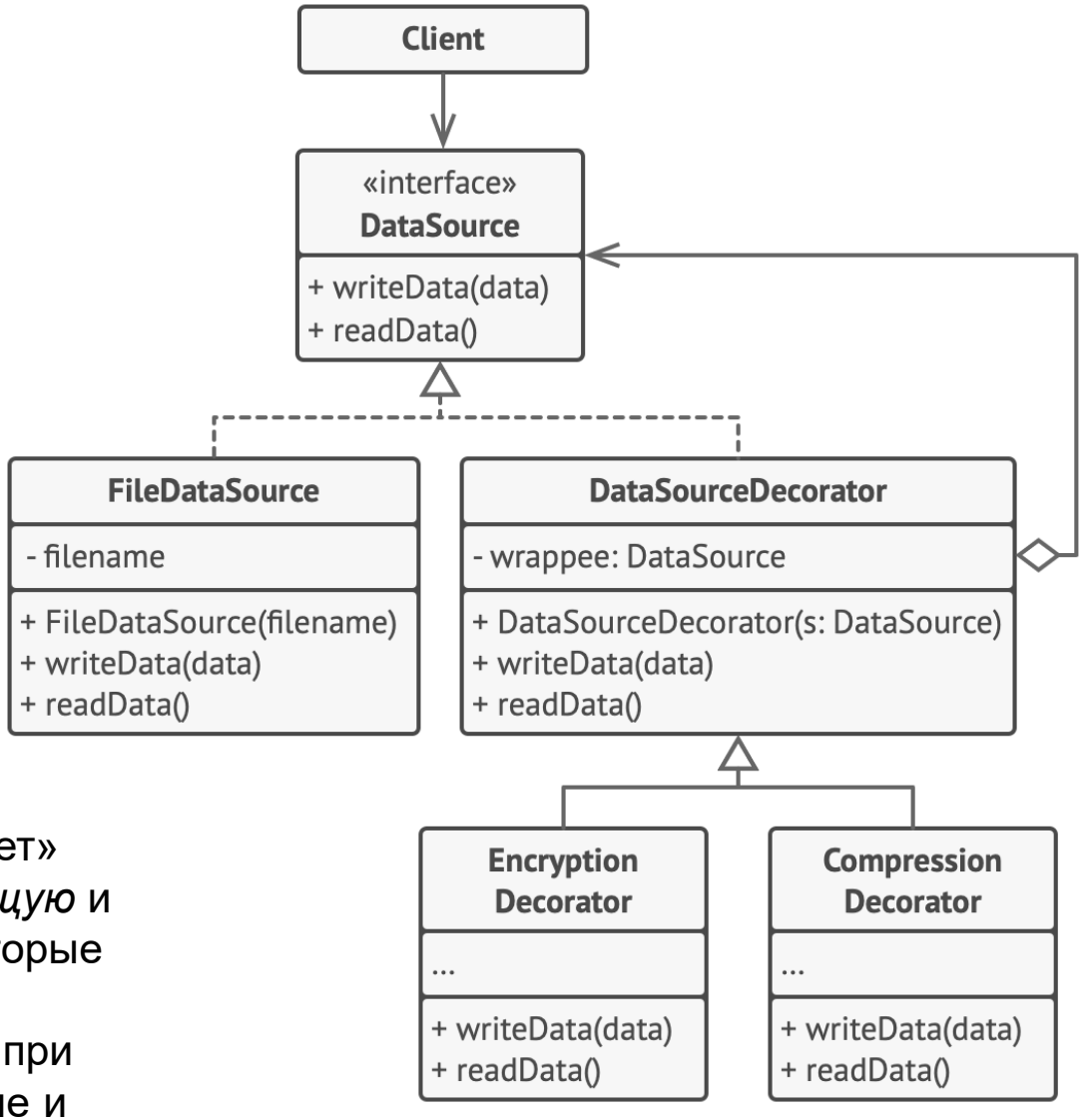


[5]**Клиент** может оборачивать простые компоненты и декораторы в другие декораторы, работая со всеми объектами через общий интерфейс компонентов.

[3]**Базовый декоратор** хранит ссылку на вложенный объект-компонент. Им может быть как конкретный компонент, так и один из конкретных декораторов. Базовый декоратор делегирует все свои операции вложенному объекту. Дополнительное поведение будет жить в конкретных декораторах.

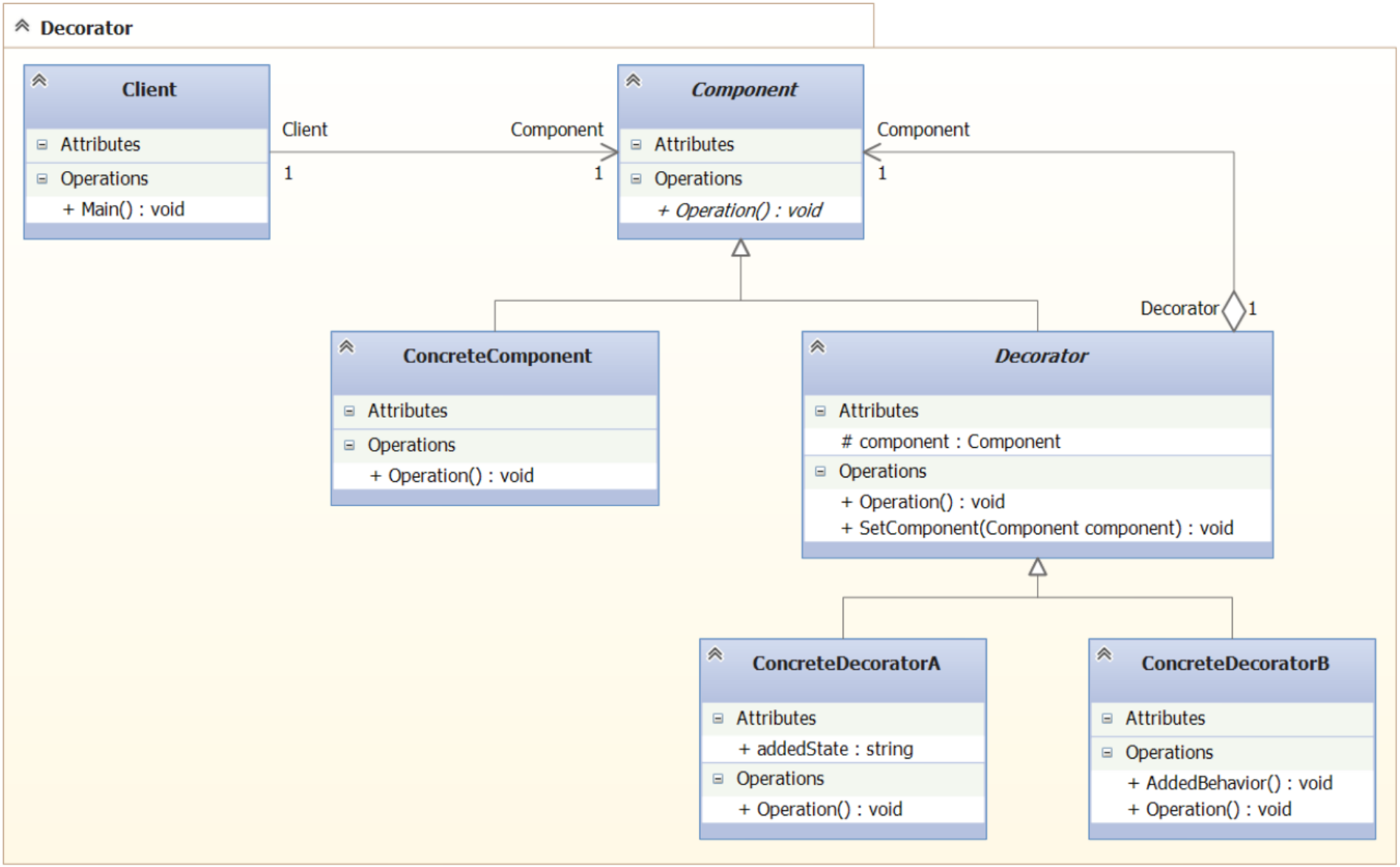
Пример

Шифрование и компрессия данных с помощью Декораторов



Приложение «оборачивает» класс данных в *шифрующую* и *сжимающую* обёртки, которые при чтении выдают оригинальные данные, а при записи — зашифрованные и сжатые.

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Component component = new ConcreteComponent();
        Decorator decoratorA = new ConcreteDecoratorA();
        Decorator decoratorB = new ConcreteDecoratorB();

        decoratorA.Component = component;
        decoratorB.Component = decoratorA;
        decoratorB.Operation();
    }
}
```

Component - **Компонент**:
Предоставляет интерфейс для объектов, которые могут быть декорированы.

```
abstract class Component
{
    public abstract void Operation();
}
```

Decorator - **Декоратор**:
Предоставляет общий интерфейс для объектов-декораторов.

ConcreteComponent - **Конкретный компонент**:
Является классом целевого объекта, который должен быть декорирован.

```
class ConcreteComponent : Component
{
    public override void Operation()
    {
        Console.WriteLine("ConcreteComponent");
    }
}
```

```
abstract class Decorator : Component
{
    public Component Component { protected get; set; }

    public override void Operation()
    {
        if (Component != null)
            Component.Operation();
    }
}
```

ConcreteDecorator - **Конкретный декоратор**:
Содержит дополнительную функциональность, которой будет расширяться целевой объект.

```
class ConcreteDecoratorA : Decorator
{
    string addedState = "Some State";

    public override void Operation()
    {
        base.Operation();
        Console.WriteLine(addedState);
    }
}
```

```
class ConcreteDecoratorB : Decorator
{
    void AddedBehavior()
    {
        Console.WriteLine("Behavior");
    }

    public override void Operation()
    {
        base.Operation();
        AddedBehavior();
    }
}
```

Отношения между участниками

Отношения между классами

- Конкретный класс ConcreteComponent связан связью отношения наследования с абстрактным классом Component.
- Абстрактный класс Decorator связан связью отношения наследования и связью отношения агрегации с абстрактным классом Component.
- Конкретные классы ConcreteDecoratorA и ConcreteDecoratorB связанны связью отношения наследования с абстрактным классом Decorator.

Отношения между объектами

- Объекты типа Decorator (ConcreteDecorator) переадресуют запросы объектам типа Component (ConcreteComponent, ConcreteDecorator) и выполняют дополнительные операции до или после переадресации.

Применение

Когда нужно добавлять обязанности объектам динамически.

Объекты помещают в «обёртки», имеющие дополнительные поведения. Обёртки и сами объекты имеют одинаковый интерфейс, поэтому для клиентов не имеет значения, с чем работать — с обычным объектом данных или с обёрнутым.

Когда нельзя расширить обязанности объекта с помощью наследования.

Во некоторых языках программирования можно запретить наследование класса (`çľǎṣṣ Bǎṣê ǧîŋǎľ`

Расширить такие классы можно только с помощью **Декоратора**.

Шаги реализации

1. **Убедиться, что в задаче есть один основной компонент и несколько опциональных** дополнений или надстроек над ним.
2. **Создать интерфейс компонента**, который описывал бы общие методы как для основного компонента, так и для его дополнений.
3. **Создать класс конкретного компонента** и поместить в него основную бизнес-логику.
4. **Создать базовый класс декораторов.** Он должен иметь поле для хранения ссылки на вложенный объект-компонент. Все методы базового декоратора должны делегировать действие вложенному объекту.
5. **Конкретный компонент, и базовый декоратор должны** следовать одному и тому же интерфейсу компонента.
6. **Создать классы конкретных декораторов,** наследуя их от базового декоратора. Конкретный декоратор должен выполнять свою добавочную функцию, а затем (или перед этим) вызывать эту же операцию обёрнутого объекта.
7. **Клиент берёт на себя ответственность за конфигурацию** и порядок обёртывания объектов.

Преимущества и недостатки

+ Большая гибкость, чем у наследования.

+ Позволяет добавлять обязанности на лету.

+ Можно добавлять несколько новых обязанностей сразу.

+ Позволяет иметь несколько мелких объектов вместо одного объекта на все случаи жизни.

- Трудно конфигурировать многократно обёрнутые объекты.

- Обилие небольших классов.